
Lab - Week 9

Replicator Dynamics

The purpose of this lab is to give you hands-on practice with implementing evolutionary game theory. The remainder of time in this lab should be used for work on your assignment with the support of the TAs..

1. The Car Park Problem

You will implement a simulation of replicator dynamics for the car park problem discussed in the seminar. Note that we will implement an individual-based or agent-based simulation rather than a system dynamics model in the first place. To make this more digestible, we can break this down into a sequence of simple steps.

Students and tutors are always trying to come to arrive for class just in time. Unfortunately, this means that everyone arrives at the car park at the same time! There are two different car parks available. Car park A is large but far away from the building. Car park B is small but close to the building. It takes only 2 minutes to walk to the tutorial rooms from car park B but 20 minutes from A. Unfortunately, it is never easy to find a parking spot straight away and the situation gets worse the more people are trying to find a spot. If n cars arrive simultaneously at the large car park A, each will suffer an n -minute delay. In the smaller car park B, each car will even suffer a delay of $5n$ -minute if n cars are simultaneously looking for a spot. How will cars be distributed across the two car parks in the long run?

Tasks

- 1.1 Decide how to represent the traits (ie. the current strategy) of the agents.
- 1.2 Decide how to represent a single game. Consider the question: Do you need individually identifiable agents?
- 1.2 Implement a function that returns the fitness for each agent in a single game. Keep the game size and the population size variable. Note: clever use of **np.where** is key here. If your representation does not lend itself to this you may want to rethink it.
Since we are dealing with a delay d but fitness needs to be higher for smaller delays and positive, you should use d^{-1} as the fitness measure. Of course, other measures are possible but they need to be monotonically decreasing in the delay.
- 1.3 Implement a function that divides the agent population into separate disjoint games. Note: **np.split** comes in handy!
- 1.4 Implement a function that evaluates the fitness for all players grouped in these games by mapping the fitness function over a list of all games.

- 1.5 Implement a function that constructs the next generation of the population by fitness-proportionate selection.
- 1.6 Use all functions from above to construct a complete Monte-Carlo simulation of replicator dynamics for the Prisoner's dilemma.
- 1.7 (Optional) Visualise the evolution of the population with a plot that displays how the proportion of agents in the two different car parks changes over time.
- 1.8 (Optional) Explore the dynamics with game sizes 10, 20 for a population size of 100. What do you observe for the players at both car parks even after long simulation times?. Does this agree with your expectations? How do you explain this effect?
- 1.9 (Optional) Can you implement the same model as a system dynamics model, i.e. solve the replicator dynamics for this scenario using differential equations? Write down the equations for the replicator dynamics and solve them numerically or analytically.
- 1.10 (Optional) Implement the same problem using cross learning? Compare the results to your replicator dynamics solution.